

UNIVERZA ALMA MATER EUROPAEA
Evropski center, Maribor

Zaključna naloga pri predmetu Odprta koda

SPLETNE IN INFORMACIJSKE TEHNOLOGIJE

Izdelava aplikacije v Android Studiu

Mentor: Dušan Fugina

Avtor: Katja Černjavič

Maribor, junij 2026

POVZETEK

V projektni nalogi je predstavljena aplikacija za shranjevanje in pregledovanje receptov, izdelana v programskem okolju Android Studio z uporabo programskega jezika Kotlin. Aplikacija omogoča dodajanje, pregledovanje, iskanje in brisanje receptov. Prav tako pa omogoča pridobivanje naključnega recepta preko API-ja. Podatki se s pomočjo SharedPreferences shranjujejo lokalno.

Ključne besede: Kotlin, Android Studio, Jetpack Compose, SharedPreferences, API.

ABSTRACT

This project presents an application for storing and managing recipes, developed in Android Studio using the Kotlin programming language. The application allows users to add, view, search, and delete recipes. It also enables users to retrieve a random recipe through an API. Data is stored locally using SharedPreferences.

Keywords: Kotlin, Android Studio, Jetpack Compose, SharedPreferences, API.

KAZALO VSEBINE

1	UVOD.....	4
2	GLAVNE FUNKCIONALNOSTI.....	5
3	OPIS KODE IN DELOVANJE APLIKACIJE.....	5
3.1	Glavna funkcija aplikacije.....	5
3.2	Začetni zaslon – DomovScreen.....	6
3.3	Seznam receptov – SeznamScreen	8
3.4	Dodajanje recepta – DodajScreen.....	9
3.5	Podrobnosti recepta – PodrobnostiScreen	11
3.6	Naključni recept iz API-ja – ApiScreen	13
3.7	Shranjevanje podatkov - SharedPreferences.....	15
4	ZAKLJUČEK.....	16
5	SEZNAM LITERATURE IN VIROV	17

KAZALO SLIK

Slika 1:	Funkcija AplikacijaRecepti()	6
Slika 2:	Koda funkcije DomovScreen().....	7
Slika 3:	Začetni zaslon aplikacije	7
Slika 4:	Koda funkcije SeznamScreen()	8
Slika 5:	Seznam receptov	9
Slika 6:	Koda funkcije DodajScreen().....	10
Slika 7:	Zaslon za dodajanje recepta	11
Slika 8:	Zaslon s podrobnostmi recepta.....	12
Slika 9:	Koda funkcije PodrobnostiScreen()	12
Slika 10:	Vmesnik MealApi	13
Slika 11:	Podatkovna razreda RandomMealResponse in Meal.....	13
Slika 12:	Koda funkcije ApiScreen()	14
Slika 13:	Zaslon za prikaz naključnega recepta.....	15
Slika 14:	Funkcija shraniRecepte() in naloziRecepte()	16

1 UVOD

Za projektno nalogo sem izdelala aplikacijo za shranjevanje in pregledovanje receptov v programskem okolju Android Studio, v programskem jeziku Kotlin. Za izdelavo uporabniškega vmesnika sem uporabila orodje Jetpack Compose.

Cilj naloge je bil razviti uporabno aplikacijo, ki prikazuje uporabo različnih programerskih konceptov, ki smo jih spoznali pri predmetu. Aplikacija uporabniku omogoča dodajanje receptov, pregled seznama že shranjenih receptov, iskanje med shranjenimi recepti, ogled recepta ter brisanje iz seznama. Omogoča pa tudi pridobivanje naključnega recepta iz spletnega API-ja.

Pri izdelavi sem uporabila delo s spremenljivkami, seznamami, funkcijami, uporabniškim vmesnikom Jetpack Compose, lokalnim shranjevanjem podatkov ter pridobivanjem podatkov iz spletnega API-ja.

2 GLAVNE FUNKCIONALNOSTI

Aplikacija je sestavljena iz petih zaslonov, ki omogočajo pregledovanje in dodajanje receptov ter pridobivanje podatkov iz spletnega API-ja.

Glavna funkcionalnost je upravljanje receptov. Uporabnik lahko dodaja nove recepte, pregleduje seznam shranjenih receptov, išče med recepti ter si lahko ogleda posamezni recept. Vsak recept vsebuje ime, sestavine, postopek priprave, čas priprave, kategorijo, nekateri pa tudi sliko.

Pri dodajanju recepta aplikacija preveri, ali so izpolnjena vsa polja. V primeru, da eno polje ni izpolnjeno, aplikacija prepreči shranjevanje podatkov. Uporabnik lahko po želji ob dodajanju recepta tudi izbere sliko iz galerije. Za prikaz receptov je uporabljena komponenta *LazyColumn*, ki omogoča učinkovito prikazovanje elementov seznama. Aplikacija omogoča tudi filtriranje receptov. Podatki o receptih se shranjujejo lokalno s pomočjo *SharedPreferences*, zato so na voljo tudi ob ponovnem zagonu aplikacije.

Aplikacija vključuje tudi povezavo s spletnim API-jem *TheMealDB*. Ob odprtju zaslona za naključni recept aplikacija s pomočjo knjižnice *Retrofit* pridobi podatke o receptu, kot so ime, kategorijo, sestavine, postopek priprave in sliko.

3 OPIS KODE IN DELOVANJE APLIKACIJE

3.1 Glavna funkcija aplikacije

Osrednji del aplikacije predstavlja funkcija *AplikacijaRecepti()*, ki skrbi za prikaz zaslonov in upravljanje podatkov. V funkciji se ustvarijo spremenljivke stanja, ki določajo, kateri zaslon ali recept je trenutno prikazan. Za preklapljanje med zasloni je uporabljena spremenljivka *trenutniZaslon*, ki vsebuje ime trenutno odprtega zaslona. Glede na vrednost spremenljivke se s stavkom *when* prikaže funkcija za posamezni zaslon.

V funkciji se prav tako naložijo že shranjeni recepti iz lokalnega pomnilnika. Recepti so shranjeni v seznamu *mutableStateListOf*, to pa omogoča samodejno posodabljanje uporabniškega vmesnika ob dodajanju ali brisanju receptov iz seznama. Ob izbiri recepta se izbrani objekt shrani v spremenljivko *izbranRecept*, ki se nato uporabi za prikaz podrobnosti recepta.

```

// Glavna funkcija aplikacije
1 Usage
@Composable
fun AplikacijaRecepti() {
    val context = LocalContext.current
    // Spremenljivka določa trenutno prikazan zaslon
    var trenutniZaslon by remember { mutableStateOf("domov") }
    // Tukaj se shrani recept, ki ga uporabnik izbere
    var izbranRecept by remember { mutableStateOf<Recept?>(null) }
    // Ob zagonu aplikacije se naložijo shranjeni recepti
    val recepti = remember {
        val nalozeniRecepti = naloziRecepte(context)
        mutableStateListOf(*nalozeniRecepti.toTypedArray())
    }

    when (trenutniZaslon) {
        "domov" -> DomovScreen(
            naRecepte = { trenutniZaslon = "seznam" },
            naDodaj = { trenutniZaslon = "dodaj" },
            naApi = { trenutniZaslon = "api" }
        )

        "seznam" -> SeznamScreen(
            recepti = recepti,
            nazaj = { trenutniZaslon = "domov" },
            izberiRecept = { it: Recept
                izbranRecept = it
                trenutniZaslon = "podrobnosti"
            }
        )

        "podrobnosti" -> PodrobnostiScreen(
            recept = izbranRecept,
            nazaj = { trenutniZaslon = "seznam" },
            izbrisi = {
                if (izbranRecept != null) {
                    recepti.remove(izbranRecept)
                    shraniRecepte(context, recepti)
                    izbranRecept = null
                    trenutniZaslon = "seznam"
                }
            }
        )

        "dodaj" -> DodajScreen(
            nazaj = { trenutniZaslon = "domov" },
            shrani = { it: Recept
                recepti.add(it)
                shraniRecepte(context, recepti)
                trenutniZaslon = "seznam"
            }
        )

        "api" -> ApiScreen(
            nazaj = { trenutniZaslon = "domov" }
        )
    }
}

```

Slika 1: Funkcija AplikacijaRecepti()

3.2 Začetni zaslon – DomovScreen

Začetni zaslon je prvi zaslon, ki se odpre ob zagonu aplikacije. Na zaslonu se prikaže naslov ter trije gumbi, ki omogočajo dostop do vseh glavnih funkcionalnosti aplikacije.

Prvi gumb uporabnika preusmeri na seznam vseh shranjenih receptov. Drugi gumb odpre zaslon za dodajanje novega recepta, tretji pa odpre zaslon za pridobivanje naključnega recepta iz spletnega API-ja.

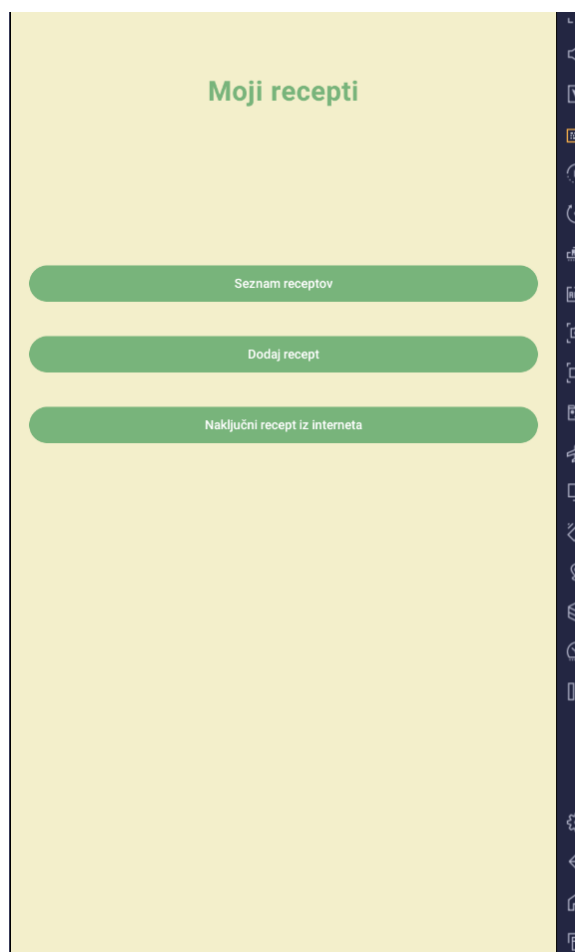
Za izdelavo zaslona sta uporabljeni komponenti *Column* in *Button*. S pritiskom na posamezni gumb se spremeni vrednost spremenljivke *trenutniZaslon*, to pa omogoči prehod na ustrezen zaslon aplikacije.

```

// Začetni zaslon
1 Usage
@Composable
fun DomovScreen(
    naRecepte: () -> Unit,
    naDodaj: () -> Unit,
    naApi: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Ozadje)
            .statusBarsPadding()
            .padding(20.dp),
        verticalArrangement = Arrangement.Top,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        this: ColumnScope
        Spacer(modifier = Modifier.height(45.dp))
        MojNaslov("Moji recepti", 30)
        Spacer(modifier = Modifier.height(170.dp))
        MojGumb("Seznam receptov", naRecepte)
        Spacer(modifier = Modifier.height(30.dp))
        MojGumb("Dodaj recept", naDodaj)
        Spacer(modifier = Modifier.height(30.dp))
        MojGumb("Naključni recept iz interneta", naApi)
    }
}

```

Slika 2: Koda funkcije DomovScreen()



Slika 3: Začetni zaslon aplikacije

3.3 Seznam receptov – SeznamScreen

Funkcija *SeznamScreen* je namenjena prikazu vseh shranjenih receptov. Na vrhu je polje za iskanje, pod njim pa je prikazan seznam.

Za filtriranje je uporabljen seznam *filtriraniRecepti*, ki s pomočjo funkcije *filter* preveri ali vneseno besedilo ustreza imenu recepta. Seznam se samodejno posodobi ob vsaki spremembi v iskalnem polju.

Za prikaz receptov je uporabljena komponenta *LazyColumn*, ki omogoča prikazovanje večjega števila elementov. Vsak recept je prikazan v elementu *Card*, kjer se poleg imena recepta vidi še kategorija, ter slika, če je bila dodana v recept. Uporabnik lahko v iskalno polje vnese besedilo, aplikacija pa sproti filtrira recepte. S klikom na posamezni recept se objekt shrani v spremenljivko *izbranRecept*, nato pa se odpre zaslon s podrobnostmi izbranega recepta. Na dnu pa se nahaja gumb *Nazaj*, ki uporabnika vrne na začetni zaslon.

```
// Zaslon s seznamom receptov
@Usage
@Composable
fun SeznamScreen(
    recepti: List<Recept>,
    nazaj: () -> Unit,
    izberiRecept: (Recept) -> Unit
) {
    var iskanje by remember { mutableStateOf("") }

    // Iskanje receptov po imenu, sestavinah ali kategoriji
    val filtriraniRecepti = recepti.filter { it: Recept ->
        it.ime.contains(iskanje, ignoreCase = true)
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(0xadffe)
            .statusBarsPadding()
            .padding(16.dp)
    ) {
        this: ColumnScope
        MojNaslov("Seznam receptov")

        Spacer(modifier = Modifier.height(10.dp))

        Vnos(
            vrednost = iskanje,
            sprememba = { iskanje = it },
            napis = "Iskaj recept"
        )

        Spacer(modifier = Modifier.height(10.dp))

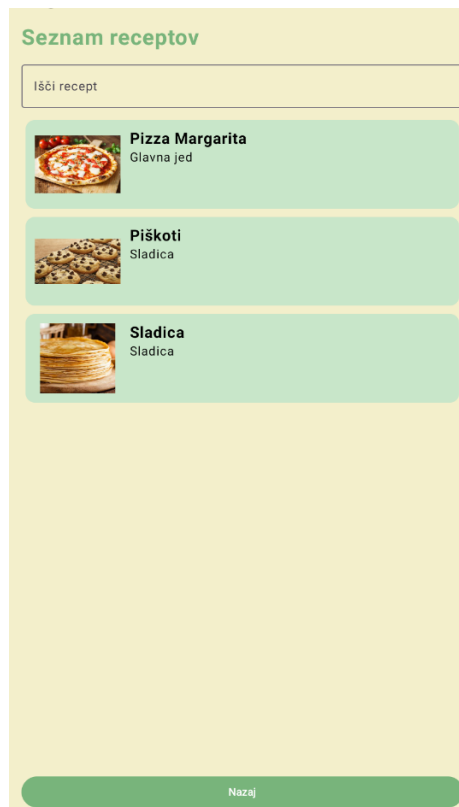
        // Prikaz receptov v seznamu
        LazyColumn(
            modifier = Modifier.weight(1f)
        ) {
            this: LazyListScope
            items(filtriraniRecepti) { this: LazyItemScope -> recept ->
                Card(
                    colors = CardDefaults.cardColors(containerColor = KarticaBarva),
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(5.dp)
                        .clickable {
                            izberiRecept(recept)
                        }
                ) {
                    this: ColumnScope
                    Row(
                        modifier = Modifier.padding(12.dp)
                    ) {
                        this: RowScope
                        if (recept.slikaUri.isNotEmpty()) {
                            Image(
                                painter = rememberAsyncImagePainter(Uri.parse(recept.slikaUri)),
                                contentDescription = null,
                                contentScale = ContentScale.Fit,
                                modifier = Modifier
                                    .width(110.dp)
                                    .height(90.dp)
                            )

                            Spacer(modifier = Modifier.width(12.dp))

                            Column {
                                this: ColumnScope
                                Text(
                                    text = recept.ime,
                                    fontSize = 20.sp,
                                    fontWeight = FontWeight.Bold
                                )
                                Text(text = recept.kategorija)
                            }
                        }
                    }
                }
            }
        }

        MojGumb("Nazaj", nazaj)
    }
}
```

Slika 4: Koda funkcije SeznamScreen()



Slika 5: Seznam receptov

3.4 Dodajanje recepta – DodajScreen

Funkcija *DodajScreen* omogoča uporabniku vnos in shranjevanje novega recepta v aplikaciji. Za posamezna polja za vnos recepta so uporabljene spremenljivke stanja, ki so ustvarjene z uporabo *remember* in *mutableStateOf*. Pred shranjevanjem aplikacija preveri, če so izpolnjena vsa polja. V primeru, da je katero polje prazno, se izpiše opozorilo in shranjevanje ni mogoče.

Uporabnik lahko doda tudi sliko iz galerije naprave. Za izbiro slike je uporabljen *ActivityResultContracts.OpenDocument()*, ki omogoča dostop do slik. Izbrana slika se shrani v spremenljivko *slikaUri* in se nato prikaže na zaslonu.

Ob uspešnem vnosu se ustvari nov objekt razreda *Recept*, ki se doda v seznam receptov. Podatki se s funkcijo *shraniRecepte()* shranijo v lokalni pomnilnik.

```

@Composable
fun DodajScreen(
    nazaj: () -> Unit,
    shrani: (Recept) -> Unit
) {
    var ime by remember { mutableStateOf("") }
    var sestavina by remember { mutableStateOf("") }
    var postopek by remember { mutableStateOf("") }
    var cas by remember { mutableStateOf("") }
    var kategorija by remember { mutableStateOf("") }
    var napaka by remember { mutableStateOf("") }
    var slikaUri by remember { mutableStateOf("") }

    val context = LocalContext.current

    // Izbrisa slika iz galerije
    val launcher = remember { LauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) } { uri ->
        if (uri != null) {
            // Dovoljenje, da slika ostane dostopna tudi po ponovnem zagonu
            context.contentResolver.takePersistableUriPermission(
                uri,
                Intent.FLAG_GRANT_READ_URI_PERMISSION
            )
            slikaUri = uri.toString()
        }
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Orzadje)
            .statusBarsPadding()
            .padding(10.dp)
    ) {
        // this: ColumnScope
        MojNaslov("Dodaj recept")

        Spacer(modifier = Modifier.height(10.dp))

        Vnos(ime, { ime = it }, "Ime recepta")
        Vnos(sestavina, { sestavina = it }, "Sestavina")
        Vnos(postopek, { postopek = it }, "Postopek")
        Vnos(cas, { cas = it }, "Cas priprave")
        Vnos(kategorija, { kategorija = it }, "Kategorija")

        Spacer(modifier = Modifier.height(10.dp))

        MojGumb(
            besedilo = "Izberi slika iz galerije",
            klik = {
                launcher.launch(arrayOf("image/*"))
            }
        )

        Spacer(modifier = Modifier.height(10.dp))

        if (slikaUri.isNotEmpty()) {
            Image(
                painter = rememberAsyncImagePainter(Uri.parse(slikaUri)),
                contentDescription = null,
                contentScale = ContentScale.Fit,
                modifier = Modifier
                    .fillMaxWidth()
                    .height(180.dp)
            )
        }

        Spacer(modifier = Modifier.height(10.dp))

        if (napaka.isNotEmpty()) {
            Text(
                text = napaka,
                color = Color.Red,
                fontWeight = FontWeight.Bold
            )
        }

        Spacer(modifier = Modifier.height(10.dp))

        MojGumb(
            besedilo = "Shrani recept",
            klik = {
                // Preverjanje, ali so vsa polja izpolnjena
                if (
                    ime.isBlank() ||
                    sestavina.isBlank() ||
                    postopek.isBlank() ||
                    cas.isBlank() ||
                    kategorija.isBlank()
                ) {
                    napaka = "Izpolnite vsa polja!"
                } else {
                    val novRecept = Recept(
                        ime = ime,
                        sestavina = sestavina,
                        postopek = postopek,
                        casPriprave = cas,
                        kategorija = kategorija,
                        slikaUri = slikaUri
                    )
                    napaka = ""
                    shrani(novRecept)
                }
            }
        )

        Spacer(modifier = Modifier.height(10.dp))

        MojGumb("Nazaj", nazaj)
    }
}

```

Slika 6: Koda funkcije DodajScreen()

Dodaj recept

Ime recepta

Sestavine

Postopek

Čas priprave

Kategorija

Izberi sliko iz galerije

Shrani recept

Nazaj

Slika 7: Zaslون za dodajanje recepta

3.5 Podrobnosti recepta – PodrobnostiScreen

Funkcija PodrobnostiScreen je namenjena prikazu podatkov izbranega recepta. Iz objekta recept, ki je bil shranjen v spremenljivko *izbranRecept*, se pridobijo podatki in se izpišejo na zaslon (ime, kategorija, čas priprave, sestavine, postopek). Če je v receptu še slika, se ta prikaže s pomočjo komponente *Image* na vrhu zaslona. Za prikaz daljših vsebin je uporabljen *verticalScroll*, ki omogoča pomikanje po zaslonu.

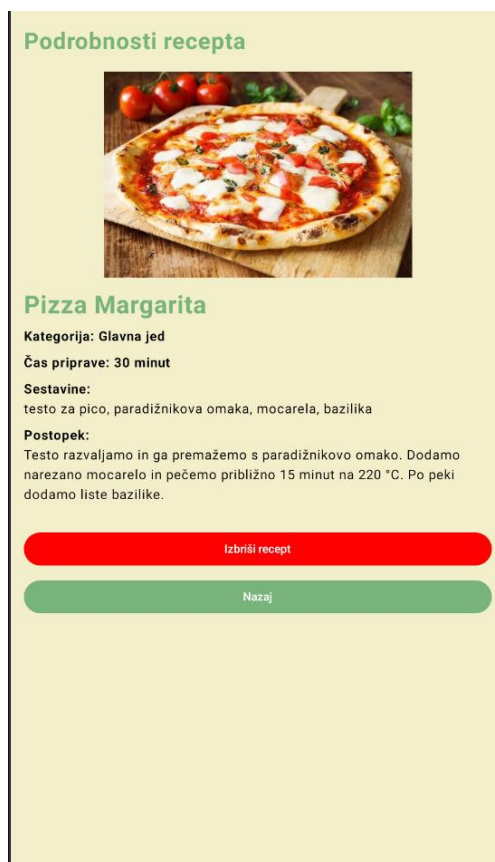
Na dnu se nahajata gumba za nazaj ter za izbris recepta. S pritiskom gumba Izbriši recept se izbran recept izbriše iz seznama receptov ter iz lokalno shranjenih podatkov. Z gumbom Nazaj pa se vrnemo na seznam receptov.

```

@Composable
fun PodrobnostiScreen(
    recept: Recept,
    nazaj: () -> Unit,
    izbriši: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(0xadffe0)
            .statusBarsPadding()
            .padding(16.dp)
            .verticalScroll(rememberScrollState())
    ) {
        this: ColumnScope
        MojNaslov("Podrobnosti recepta")
        Spacer(modifier = Modifier.height(20.dp))
        if (recept != null) {
            if (recept.slikaUri.isNotEmpty()) {
                Image(
                    painter = rememberAsyncImagePainter(Uri.parse(recept.slikaUri)),
                    contentDescription = null,
                    contentScale = ContentScale.Fit,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(250.dp)
                )
                Spacer(modifier = Modifier.height(15.dp))
            }
            Text(
                text = recept.ime,
                fontSize = 30.sp,
                fontWeight = FontWeight.Bold,
                color = WaslovBarva
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text("Kategorija: ${recept.kategorija}", fontWeight = FontWeight.Bold)
            Spacer(modifier = Modifier.height(8.dp))
            Text("Čas priprave: ${recept.casPriprave}", fontWeight = FontWeight.Bold)
            Spacer(modifier = Modifier.height(8.dp))
            Text("Sestavine:", fontWeight = FontWeight.Bold)
            Text(recept.sestavine)
            Spacer(modifier = Modifier.height(8.dp))
            Text("Postopek:", fontWeight = FontWeight.Bold)
            Text(recept.postopek)
        }
        Spacer(modifier = Modifier.height(30.dp))
        MojGumb("Izbriši recept", izbriši, Color.Red)
        Spacer(modifier = Modifier.height(10.dp))
        MojGumb("Nazaj", nazaj)
    }
}

```

Slika 9: Koda funkcije PodrobnostiScreen()



Slika 8: Zaslonski prikaz s podrobnostmi recepta

3.6 Naključni recept iz API-ja – ApiScreen

API predstavlja vmesnik, ki aplikaciji omogoča pridobivanje podatkov iz spletne storitve. Funkcija *ApiScreen* omogoča pridobivanje naključnega recepta iz spletnega API-ja TheMealDB.

Za komunikacijo z API-jem je uporabljena knjižnica *Retrofit*. Ob odprtju zaslona za naključni recept se izvede zahteva za pridobivanje podatkov naključnega recepta. Podatki o imenu, kategoriji, sestavinah, postopku priprave in povezava do slike se shranijo v spremenljivke.

Za izvedbo zahteve je uporabljena funkcija *LaunchedEffect*, ki se izvede ob odprtju zaslona. Podatki pa se pridobijo preko funkcije *getRandomMeal()*, ki je definirana v vmesniku *MealApi*. Po uspešno prejetih podatkih se ti prikažejo na zaslonu, v primeru napake pa se uporabniku izpiše obvestilo.

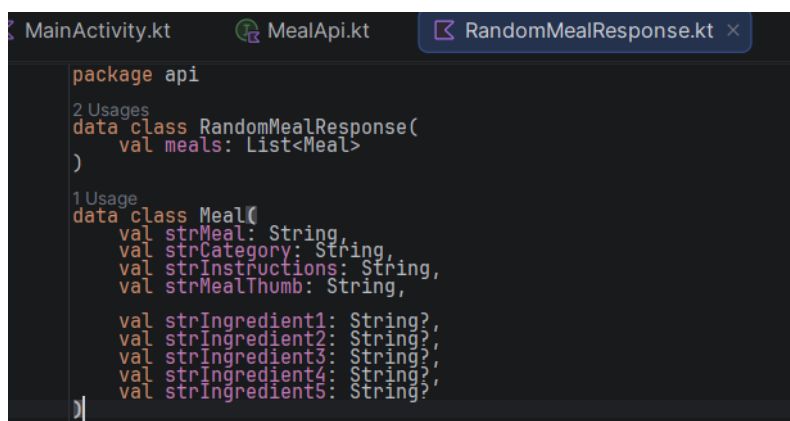
Za pravilno obdelavo podatkov iz API-ja sta uporabljena podatkovna razreda *RandomMealResponse* in *Meal*. Prvi vsebuje seznam pridobljenih receptov, drugi pa podatke o posameznem receptu.



```
package api
import retrofit2.http.GET
import api.RandomMealResponse

2 Usages
interface MealApi {
    @GET("random.php")
    suspend fun getRandomMeal(): RandomMealResponse
}
```

Slika 10: Vmesnik MealApi



```
package api
2 Usages
data class RandomMealResponse(
    val meals: List<Meal>
)
1 Usage
data class Meal(
    val strMeal: String,
    val strCategory: String,
    val strInstructions: String,
    val strMealThumb: String,

    val strIngredient1: String?,
    val strIngredient2: String?,
    val strIngredient3: String?,
    val strIngredient4: String?,
    val strIngredient5: String?
)
```

Slika 11: Podatkovna razreda RandomMealResponse in Meal

```

// Zaslon za naključni recept iz API-ja
@Composable
fun ApiScreen(
    nazaj: () -> Unit
) {
    var ime by remember { mutableStateOf("") }
    var kategorija by remember { mutableStateOf("") }
    var sestavina by remember { mutableStateOf("") }
    var postopek by remember { mutableStateOf("") }
    var slika by remember { mutableStateOf("") }
    var napaka by remember { mutableStateOf("") }
    var nalagam by remember { mutableStateOf(true) }

    // API se pokliče samo enkrat, ko se zaslon odpre
    LaunchedEffect(Unit) {
        try {
            val odgovor = RetrofitInstance.api.getRandomMeal()
            val recept = odgovor.meals.first()

            ime = recept.strMeal
            kategorija = recept.strCategory
            postopek = recept.strInstructions
            slika = recept.strMealThumb

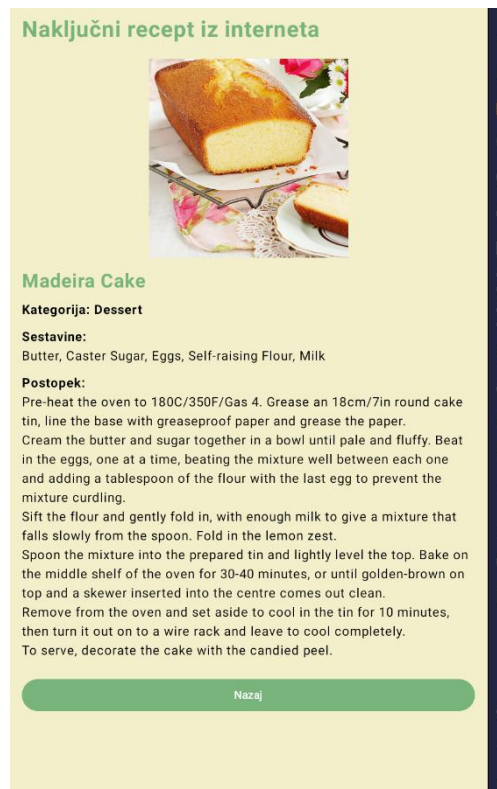
            sestavina = listOf(
                recept.strIngredient1,
                recept.strIngredient2,
                recept.strIngredient3,
                recept.strIngredient4,
                recept.strIngredient5
            )
                .filter { !it.isNullOrEmpty() }
                .joinToString(", ")

            napaka = ""
            nalagam = false
        } catch (e: Exception) {
            napaka = "Napaka pri pridobivanju podatkov."
            nalagam = false
        }
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(0xadffe0)
            .statusBarsPadding()
            .padding(16.dp)
    ) {
        this: ColumnScope
        MojNaslov("Naključni recept iz interneta")
        Spacer(modifier = Modifier.height(20.dp))
        if (nalagam) {
            Text(
                text = "Pridobivam recept",
                fontWeight = FontWeight.Bold
            )
        }
        if (napaka.isNotEmpty()) {
            Text(
                text = napaka,
                color = Color.Red,
                fontWeight = FontWeight.Bold
            )
        }
        if (ime.isNotEmpty()) {
            if (slika.isNotEmpty()) {
                Image(
                    painter = rememberAsyncImagePainter(slika),
                    contentDescription = null,
                    contentScale = ContentScale.Fit,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(250.dp)
                )
                Spacer(modifier = Modifier.height(15.dp))
            }
            Text(
                text = ime,
                fontSize = 24.sp,
                fontWeight = FontWeight.Bold,
                color = NaslovBarva
            )
            Spacer(modifier = Modifier.height(10.dp))
            Text("Kategorija: $kategorija", fontWeight = FontWeight.Bold)
            Spacer(modifier = Modifier.height(10.dp))
            Text("Sestavina:", fontWeight = FontWeight.Bold)
            Text(text = sestavina)
            Spacer(modifier = Modifier.height(10.dp))
            Text("Postopek:", fontWeight = FontWeight.Bold)
            Text(text = postopek)
        }
        Spacer(modifier = Modifier.height(20.dp))
        MojGumb("Nazaj", nazaj)
    }
}

```

Slika 12: Koda funkcije ApiScreen()



Slika 13: Zaslona za prikaz naključnega recepta

3.7 Shranjevanje podatkov - SharedPreferences

Za shranjevanje receptov je uporabljeno lokalno shranjevanje podatkov s pomočjo SharedPreferences. Na ta način se podatki ohranijo po zaprtju in ponovnem zagonu aplikacije.

Funkcija *shraniRecepte()* skrbi, da so vsi recepti shranjeni v lokalni pomnilnik. Najprej se podatki pretvorijo v besedilo, potem pa se s pomočjo SharedPreferences shranijo pod ključem *vsRecepti*.

Ob zagonu aplikacije pa se izvede funkcija *naloziRecepte()*, ki iz lokalnega pomnilnika prebere shranjene podatke, jih razdeli na dele in ponovno ustvari objekte razreda *Recept*.

S pomočjo teh dveh funkcij je omogočeno ohranjanje podatkov ob ponovnem zagonu aplikacije.

```

2 Usages
fun shraniRecepte(
    context: Context,
    recepti: List<Recept>
) {
    val preferences =
        context.getSharedPreferences("Recepti", Context.MODE_PRIVATE)

    val podatki = recepti.joinToString("\n")

    preferences.edit()
        .putString("vsiRecepti", podatki)
        .apply()
}

// Nalaganje shranjenih receptov iz SharedPreferences
1 Usage
fun naloziRecepte(
    context: Context
): MutableList<Recept> {
    val preferences =
        context.getSharedPreferences("Recepti", Context.MODE_PRIVATE)

    val podatki =
        preferences.getString("vsiRecepti", "") ?: ""

    val seznam = mutableListOf<Recept>()

    if (podatki.isNotEmpty()) {
        val vrstice = podatki.split("\n")

        for (vrstica in vrstice) {
            val deli = vrstica.split("|")

            if (deli.size == 6) {
                seznam.add(
                    Recept(
                        deli[0],
                        deli[1],
                        deli[2],
                        deli[3],
                        deli[4],
                        deli[5]
                    )
                )
            }
        }
    }

    return seznam
}

```

Slika 14: Funkcija shraniRecepte() in naloziRecepte()

4 ZAKLJUČEK

Izdelava naloge mi je omogočila, da sem uporabila in utrdila znanja, pridobljena pri predmetu. Pri izdelavi sem se srečala z nekaterimi izzivi, predvsem pri shranjevanju slik in delu z API-jem. Projekt mi je pomagal pri razumevanju razvoja Android aplikacij in povezovanju teorije s praktično uporabo.

5 SEZNAM LITERATURE IN VIROV

1. Android Developers. (2025). Jetpack Compose Documentation. Pridobljeno s <https://developer.android.com/jetpack/compose>
2. TheMealDB. (2025). TheMealDB API. Pridobljeno s <https://www.themealdb.com/api.php>
3. JetBrains. (2025). Kotlin Documentation. Pridobljeno s <https://kotlinlang.org/docs/home.html>